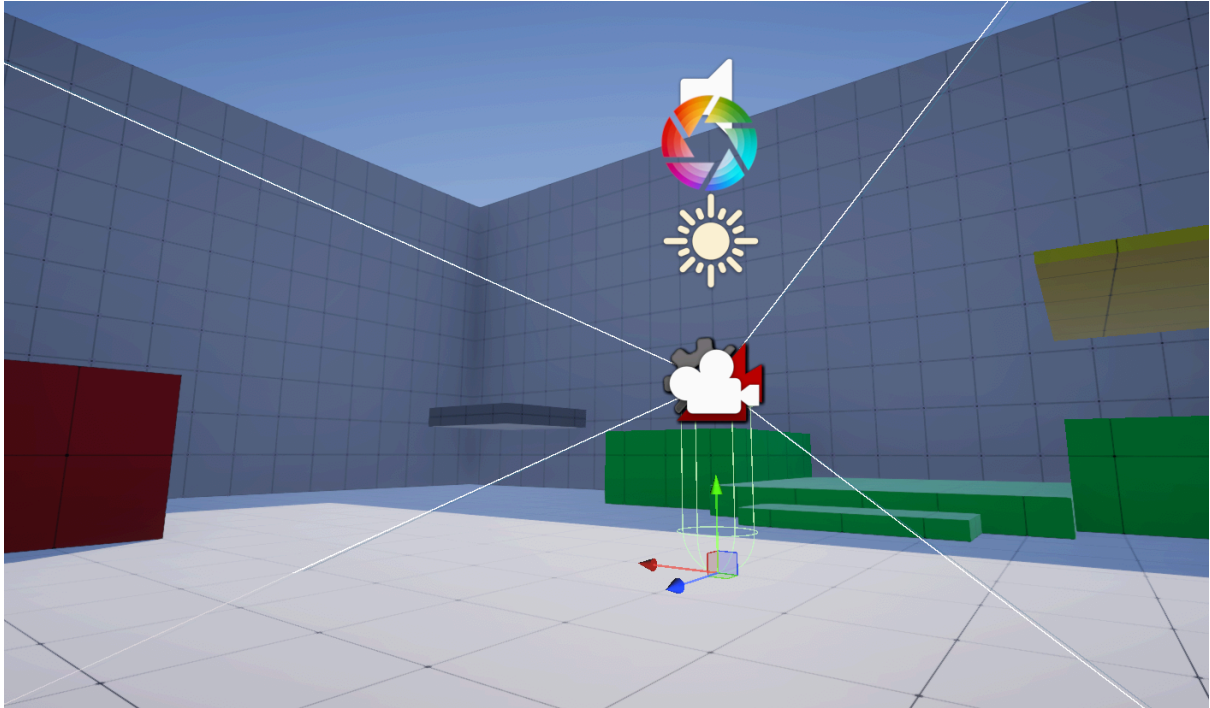


MFPC – Modular First Person Controller

Documentation v1.3



By Faber Brizio

New in v1.3 — Movement Expansion

Version 1.3 expands MFPC with new optional movement feedback and camera presentation features, while keeping the controller modular and system-agnostic.

New optional systems

- **SteepSlopeSlideModule**
Adds forced downhill sliding on slopes above a configurable angle.
- **LeanDutchModule**
Extends camera leaning through Cinemachine Dutch roll.
- **Advanced headbob blend profiles**
Improves per-state camera feel and transition polish.

Movement and camera additions

- Camera leaning improvements
- Camera tilt feedback on steep slopes
- Better movement presentation on uneven terrain
- Continued modular integration with Cinemachine-based camera workflows

Audio additions

- New **snow** surface sound layer/preset

These additions continue MFPC's philosophy of optional, production-ready modules: developers can use them, modify them, or remove them without changing the core controller architecture.

1. What is MFPC?

MFPC (Modular First Person Controller) is a clean, production-ready first person controller designed for flexibility, performance, and long-term maintainability.

It provides a solid, modern FPS movement foundation without forcing architectural decisions, gameplay systems, or project structure.

MFPC is built to integrate smoothly into existing projects as well as new prototypes, allowing developers to extend it freely instead of fighting against a rigid framework.

MFPC focuses on **doing a set of core systems extremely well**, rather than trying to solve every possible gameplay scenario.

Designed for

- First-person games and immersive experiences
- Prototyping and production environments
- Developers who want control over systems and architecture
- Projects using Cinemachine
- Both Legacy and New Input System workflows

Not included by design

- Multiplayer or networking logic
- Animation-driven movement systems
- Advanced traversal mechanics such as wall-running, vaulting, ledge climbing, etc.

MFPC is designed to **integrate with them**, not restrict or replace them. This makes MFPC suitable for both single-player and multiplayer projects, as well as animation-driven workflows.

MFPC is not animation-driven, but it is fully compatible with animation systems through optional integration components.

Key principle

MFPC is intended to be a **foundation, not a constraint**.

2. Core philosophy

MFPC is built as a modular first person controller that stays focused on movement, camera behavior, and extensibility.

It is designed to provide a production-ready controller foundation without forcing:

- a specific animation setup
- a UI or HUD
- a stamina bar
- a multiplayer architecture
- hardcoded layers, tags, or project assumptions

Version 1.2 expands this philosophy further by introducing optional gameplay modules such as stamina, while keeping the base controller usable on its own.

To help users test the stamina system quickly, MFPC includes a simple **demo-only stamina bar** in the demo scene. This script is intentionally lightweight and exists only as a sample integration. It is **not a required dependency**, and the full demo UI can be removed without changing the core controller package.

Controller-first design

Movement, camera, and player state are handled directly by the controller. No external systems are required to make the controller function.

System-agnostic

MFPC does not require:

- Animator Controllers
- UI systems
- Stamina systems
- Specific layers or tags

All of these can be added externally if needed.

Clean inspector

Only relevant and commonly tuned values are exposed. The controller avoids excessive abstraction while remaining easy to customize.

Input & Control Options

MFPC includes two controller variants:

- `PlayerController_NewInputSystem`
- `PlayerController_Legacy`

Both controllers support the same core gameplay behavior:

- walking
- sprinting
- crouching
- jumping
- aiming
- leaning
- footstep and landing audio
- Cinemachine-driven camera states

Input flexibility

MFPC supports:

- mouse and keyboard
- gamepad
- Unity Legacy Input System
- Unity New Input System

Shared control options

Both controller variants support:

- hold or toggle crouch
- hold or toggle sprint
- optional auto-unsprint when stopping
- optional lean enable/disable
- forward-only sprint behavior
- stamina-aware sprint restrictions when the stamina module is used

Cinemachine-friendly

Camera behavior is designed around Cinemachine Virtual Cameras and extensions rather than custom camera math.

Performance-safe

- No runtime allocations in Update
- No per-frame instantiation
- Designed to be GC-free during gameplay

These principles make MFPC suitable as a long-term base controller rather than a throwaway prototype asset.

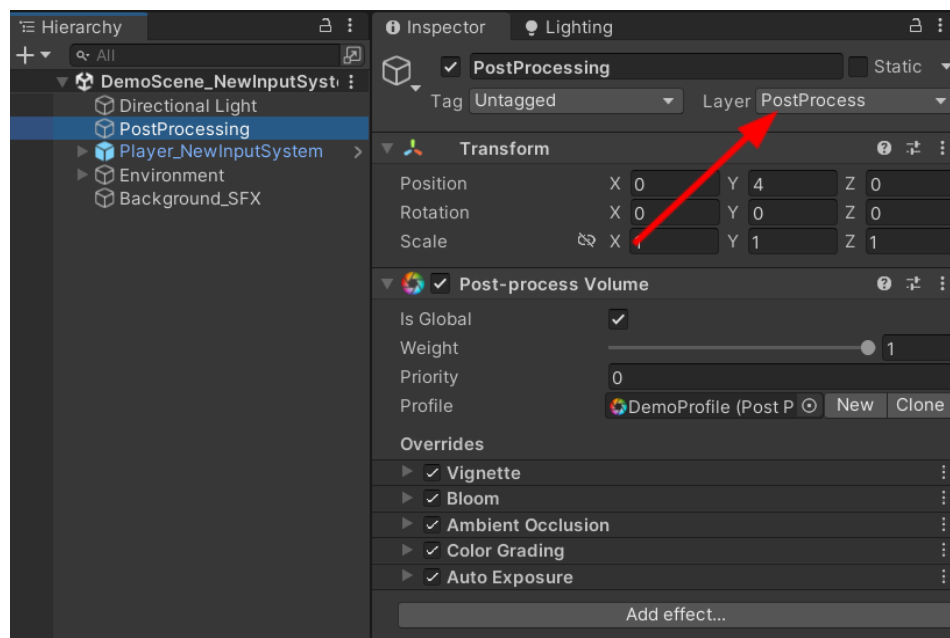
3. Quick setup (5 minutes)

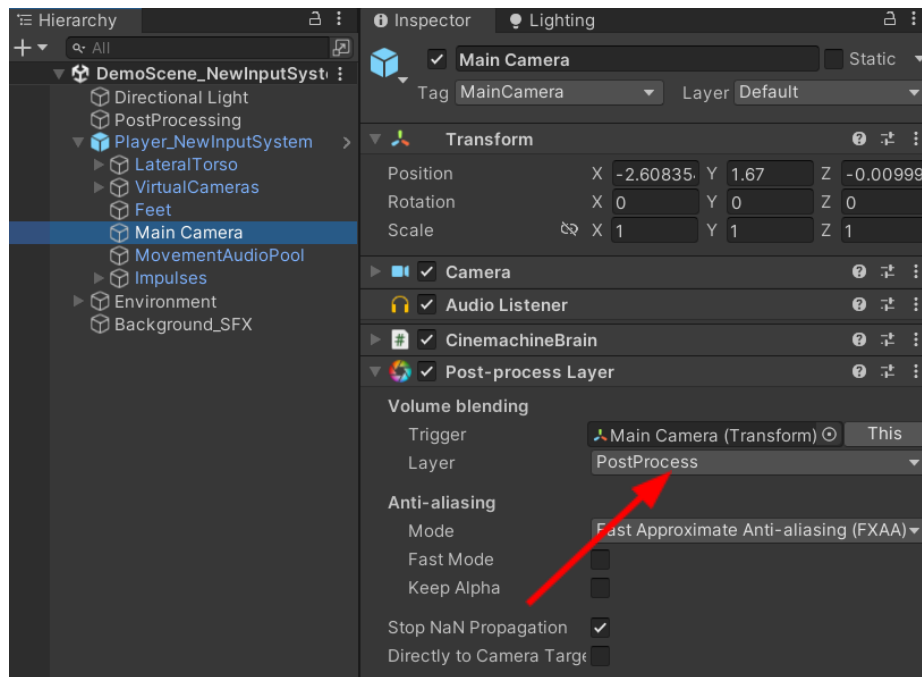
Requirements

- Unity 2021 or newer (recommended)
- Cinemachine package installed
- Input system package installed (to use the new-input-system-based character controller)

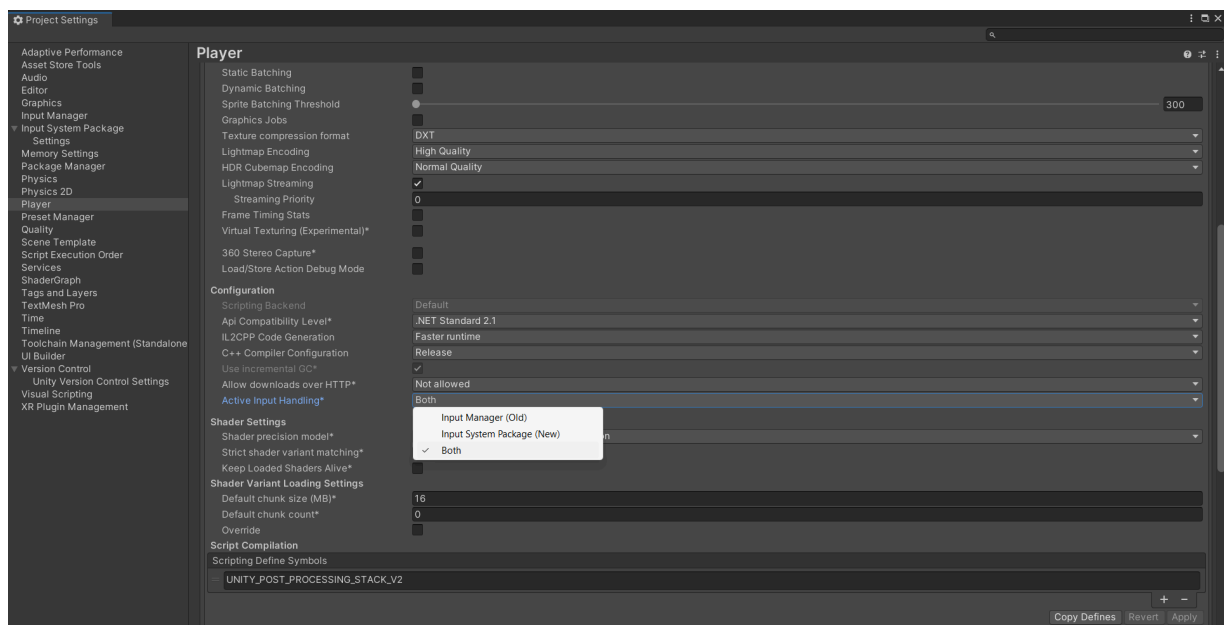
Steps

1. Import MFPC
2. Create a new scene or open an existing one, and remove the main camera (the controllers already have a main camera as a child).
3. Add one of the following prefabs:
 - PlayerController_Legacy (*Legacy Input System*)
 - PlayerController_NewInputSystem (*New Input System*)
4. Optional (for Built-in version): double check post-processing layer and assign it in the “Layer” field in the Post-process layer component.

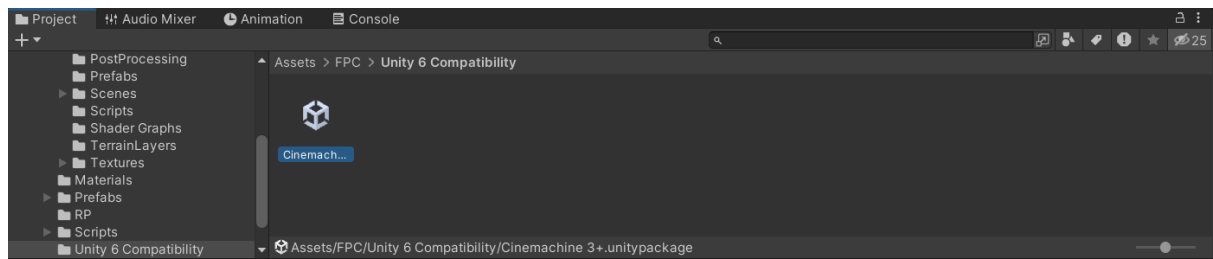




5. If you are using the **New Input** system solution, check to be using both old and new inputs or the new input system. To do that open your Project Settings > Player, under Configuration > Active Input Handling select: Both or Input System Package (New).



6. If using Unity 6.0+, you may get some compilation errors on import. That happens because of API updates in newer versions of Cinemachine (3+). To solve this issue, open the folder to Asset > FPC > Unity 6 Compatibility and click and install the update package named "Cinemachine 3+". All import errors should be solved.



7. Press Play.

No additional configuration is required to get basic movement, camera control, and audio working.

MFPC does not modify project settings, layers, or input configuration automatically.

4. Choosing an input system

MFPC provides **two separate controller implementations**, each designed for a specific input workflow.

Legacy Input System

Component: `PlayerController_Legacy`

- Uses `Input.GetAxis` and `Input.GetKey`
- Compatible with older projects
- Minimal setup required
- Best choice for legacy workflows

New Input System

Component: `PlayerController_NewInputSystem`

- Uses Unity's New Input System
- Supports mouse and gamepad input
- Includes sensitivity normalization and smoothing
- Designed to match Legacy Input feel as closely as possible

Important note

Only **one controller should be used at a time**.

Do not attach both controllers to the same player object.

Both controllers share the same movement philosophy and exposed state, making it easy to switch input systems without rewriting gameplay logic.

5. Movement & player states

MFPC provides a complete and reliable first-person movement core, covering the most common gameplay needs while remaining flexible and unopinionated.

Supported movement states

- Walking
 - Sprinting
 - Crouching
 - Jumping
 - Falling / grounded detection
 - Leaning (left / right)
 - Aiming
-

Sprint behavior

MFPC includes a robust sprint system designed to feel natural and predictable.

- Optional **hold or toggle** sprint mode
- Forward-only sprint restriction
- Sprint automatically disabled when:
 - Crouching
 - Leaning
- **Optional auto-unsprint**
 - Sprint is cancelled automatically when the player stops moving
 - Prevents “stuck sprint” states in toggle mode
 - Can be enabled or disabled via settings

Sprint state is always exposed cleanly for external systems (UI, stamina, animation, etc.).

Crouching & auto-uncrouch

Crouching is fully collision-safe and supports both gameplay styles.

- Optional **hold or toggle** crouch mode
- Smooth camera and CharacterController height transitions
- **Collision-safe auto-uncrouch**
 - Physics-based clearance check
 - Prevents standing up under low ceilings
 - Does not require specific layers or tags
 - Works during smooth crouch transitions

If uncrouching is blocked, the controller automatically remains crouched.

Leaning

- Lean left / right
- Smooth interpolation in and out
- Leaning temporarily disables sprint to avoid camera and control conflicts
- Lean amount and smoothing fully configurable

Grounding & jumping

- CharacterController-based grounding
- Tunable gravity multiplier
- Jump impulse with camera feedback
- Landing detection for:
 - Audio
 - Camera impulse
- Stable behavior on uneven terrain

MFPC intentionally avoids animation-driven movement, keeping player control responsive, deterministic, and easy to extend.

Optional Advanced Slope Handling

MFPC includes an optional slope handling module that improves stability and control on uneven terrain and inclined surfaces.

While Unity's `CharacterController` handles basic grounding, it can produce minor bouncing, unstable downhill movement, or inconsistent projection on slopes. The `SlopeHandler` refines this behavior without overriding the controller.

Ground Stick Force

When grounded and moving downward, a configurable downward force is applied:

```
[SerializeField] float stickForce = -2f;
```

This:

- Improves slope adhesion
- Prevents micro-detachment
- Stabilizes downhill movement

The force only applies when grounded and falling, ensuring jump behavior remains unaffected.

Surface-Aligned Movement

When a surface is detected below the player, movement is projected onto the slope plane:

```
movement = Vector3.ProjectOnPlane(movement, hit.normal);
```

This ensures:

- Proper incline alignment
- Natural uphill/downhill traversal
- No artificial vertical velocity injection

Detection uses a downward raycast based on the controller height and requires no special layers or tags.

Recommended Use Cases

Enable advanced slope handling if your game features:

- Natural outdoor terrain
- Large ramps or architectural inclines
- Frequent elevation transitions
- Precision-based first-person control

For flat indoor environments, the default grounding may be sufficient.

The slope handler is fully optional, modular, and keeps MFPC deterministic and responsive while improving terrain behavior in outdoor or elevation-heavy environments.

Optional Stamina Module

Version 1.2 introduces an optional stamina system through:

Component: `StaminaModule`

This module is completely optional.

If no stamina module is assigned, the controller behaves normally with no stamina restrictions.

What it controls

- stamina drain while sprinting
- stamina cost on jump

- delayed regeneration
- exhaustion state
- sprint lockout until recovery threshold is reached

Main stamina settings

- `maxStamina`
- `startingStamina`
- `sprintDrainPerSecond`
- `jumpStaminaCost`
- `regenerationDelay`
- `regenerationPerSecond`
- `staminaRecoveryThreshold`

Behavior

- sprint drains stamina only while sprint speed is actually being applied
- jumping can consume stamina immediately
- when stamina reaches zero, sprint becomes unavailable
- regeneration begins after a configurable delay
- sprint remains locked until stamina recovers past the threshold

Design goal

The stamina system is implemented as a separate module instead of being built into the controller.

This keeps MFPC flexible for projects that want stamina, and lightweight for projects that do not.

Optional Steep Slope Sliding

Version 1.3 introduces an optional steep-slope sliding module:

Component: `SteepSlopeSlideModule`

This module detects slopes above a configurable walkable angle and forces downhill sliding when the player is grounded on terrain that is too steep to stand on safely.

What it does

- Detects the ground angle beneath the player
- Compares it against a configurable maximum walkable slope angle
- Applies downhill sliding when that angle is exceeded

- Gradually ramps slide speed over time
- Keeps the controller attached to the slope using a configurable stick force
- Optionally triggers camera tilt feedback while sliding

Main settings

- `maxWalkableSlopeAngle`
- `groundProbeDistance`
- `initialSlideSpeed`
- `maxSlideSpeed`
- `timeToReachMaxSlide`
- `slideStickForce`

Optional camera feedback

If enabled, the module can also drive a Cinemachine impulse while sliding:

- `enableCameraTilt`
- `slopeTiltImpulse`
- `initialTiltImpulseVelocity`
- `maxTiltImpulseVelocity`

This helps steep-slope movement feel more physical and readable without changing the base controller logic.

Design goal

Like the stamina system and slope handler, steep-slope sliding is fully optional.

If no `SteepSlopeSlideModule` is assigned, MFPC behaves exactly as before.

6. Camera & Cinemachine integration

MFPC is built around Cinemachine virtual cameras rather than custom camera math.

Camera structure

- Uses Cinemachine Virtual Cameras
- No custom camera math or overrides
- Compatible with existing Cinemachine setups

Features

- Sprint FOV kick (via virtual camera swap)
- Aim camera variants
- Headbob using Cinemachine Noise
- Camera lean tied to player state
- Vertical rotation clamping
- Leaning through torso/neck transforms

Each movement state (idle, walk, sprint, crouch, aim) can have its own virtual camera configuration if desired.

Headbob implementation

Headbob is handled via **Cinemachine Noise**, not procedural transforms:

- Different noise profiles per movement state
- Easy to tweak without code changes
- Zero runtime allocations

This approach keeps camera behavior clean, predictable, and visually consistent.

Camera blends

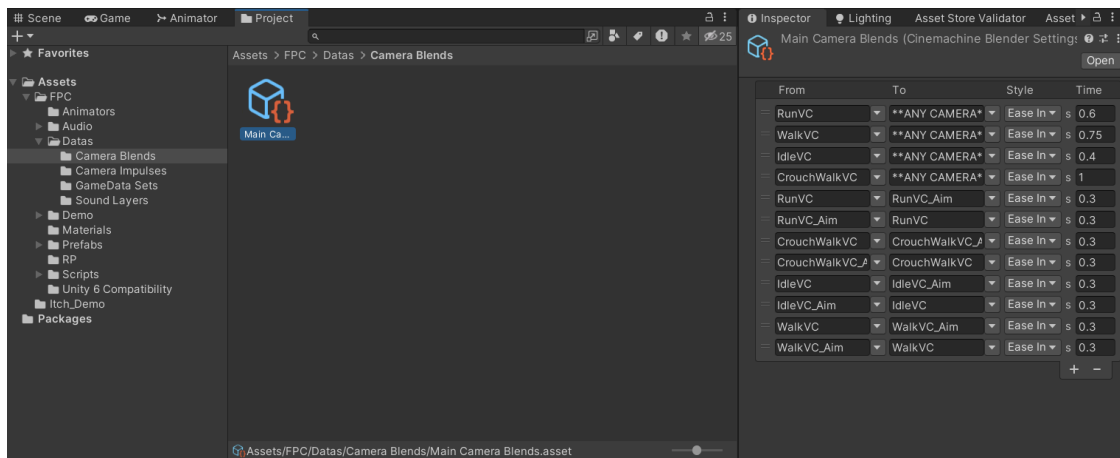
MFPC now supports **custom Cinemachine blends** between virtual camera transitions.

This allows smoother transitions such as:

- idle to walk
- walk to sprint
- walk to aim
- crouch walk to crouch aim
- sprint to aim
- return transitions between any of the above

Using a shared Cinemachine blend asset makes transitions more polished and easier to tune project-wide.

You can configure blend duration and style directly through the Cinemachine Brain custom blends asset.



Jump, Landing, and Camera Impulses

MFPC includes jump and landing feedback through:

- `jumpShakingImpulseSource`
- `landShakingImpulseSource`
- `landBumpingImpulseSource`

Jump behavior

When the player performs an actual jump:

- upward force is applied
- a jump sound can play
- jump camera shake can be triggered
- `OnJumpStart` is invoked

Landing behavior

Landing effects are triggered only after the player has remained airborne longer than a configurable threshold.

This avoids playing full landing feedback for tiny bumps or short grounded interruptions.

Landing feedback can include:

- landing sounds
- shake impulse
- bump impulse
- intensity scaled by airborne time

This gives jumps and falls more physical feedback without requiring extra animation logic.

Optional headbob disable module

MFPC uses Cinemachine Noise for headbob and movement camera shake. In some projects, developers may want to disable this behavior globally at startup without editing each virtual camera manually.

For this purpose, MFPC includes an optional helper component:

Component: `NoiseHeadbobController`

What it does:

- Searches all child `CinemachineVirtualCamera` components
- Looks for a `CinemachineBasicMultiChannelPerlin` noise module on each camera
- Disables amplitude and frequency at startup when headbob/noise is turned off in `GameData`

This is useful for:

- Accessibility options
- Runtime graphics or comfort presets
- Projects that want to keep camera structure intact while disabling procedural noise

Typical workflow:

1. Add `NoiseHeadbobController` to the player
2. Assign the `GameData` asset
3. If `EnableNoiseAndHeadbob` is false, all supported Cinemachine noise components in child virtual cameras are disabled on startup

This keeps the camera stack modular and avoids custom per-camera setup changes

Lean-based camera roll

Version 1.3 adds an optional Cinemachine-based lean roll system:

Component: `LeanDutchModule`

This module reads the controller's current lean angle and applies it to Cinemachine using Dutch roll, allowing the camera to tilt visually while leaning.

What it does

- Reads the current lean angle from either controller
- Converts that value into Cinemachine Dutch
- Applies the result to selected virtual cameras
- Supports automatic setup of a Cinemachine extension on target cameras
- Allows clamped and scaled roll intensity

Main settings

- `dutchMultiplier`
- `maxDutchAngle`
- `virtualCameras`

If no explicit virtual camera list is assigned, the module automatically targets child Cinemachine virtual cameras.

Design goal

This keeps leaning presentation modular and editor-friendly:

- the controller still handles player lean state
- Cinemachine handles the camera roll
- developers can tune or disable the effect independently

Camera tilt on steep slopes

When `SteepSlopeSlideModule` is used, MFPC can optionally generate camera tilt feedback while sliding on steep terrain.

This is handled through a Cinemachine impulse source and helps communicate:

- loss of stable footing
- increasing slide intensity
- terrain steepness over time

Because the tilt is driven as an optional module, projects can keep the sliding behavior with or without additional camera presentation.

7. Audio system

MFPC includes a flexible, surface-aware audio system designed to adapt to different environments without hardcoded logic.

Included audio features

- Footstep sounds by surface type
- Jump start and landing sounds
- Water-specific audio handling
- Audio pooling (no runtime instantiation)
- Support for both terrain layers and individual objects

Audio behavior is fully data-driven using ScriptableObjects.

Surface sound presets (SurfaceSFX)

Surface sounds are defined using the **SurfaceSFX** ScriptableObject.

Each SurfaceSFX asset contains:

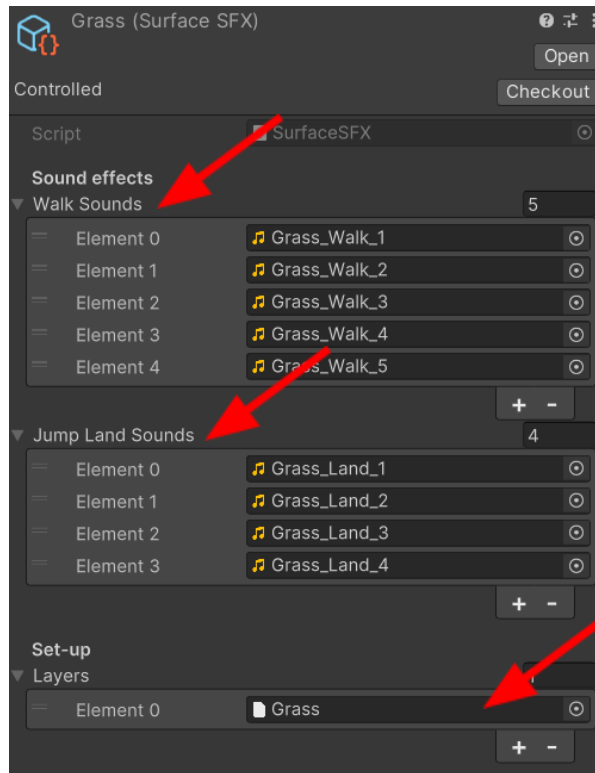
- A set of **footstep sounds**
- A set of **landing sounds**
- Optional references to **Terrain Layers**

This allows the same audio preset to be reused across multiple surfaces and scenes.

Creating a new surface sound preset

1. In the Project window, right-click
Create → Surface SFX
2. Name the asset (e.g. **Wood**, **Carpet**, **Snow**)
3. Assign:
 - Walk sounds
 - Jump / landing sounds
4. (Optional) Assign one or more **Terrain Layers**
These will automatically trigger this sound when walking on terrain using those layers.

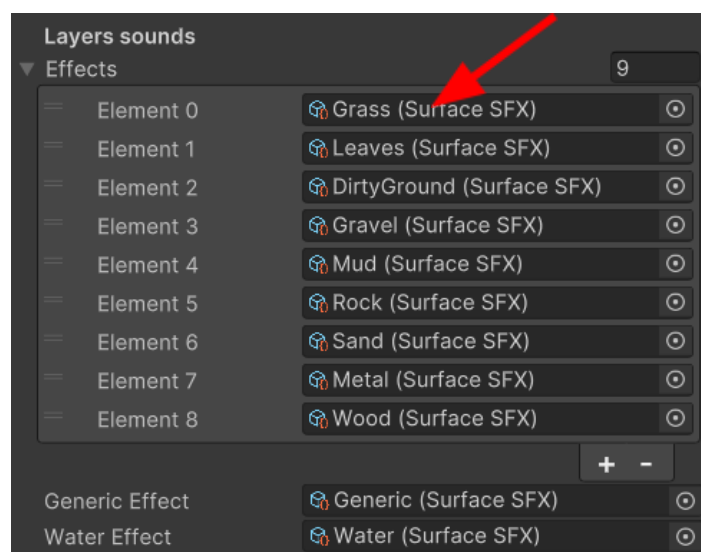
No scripting is required. Below an example Surface SFX preset is shown:



Using surface sounds on terrain

To use surface-based audio on terrain:

- After creating one or more **SurfaceSFX** assets as shown before (*friendly reminder: assign the corresponding **Terrain Layers** inside each SurfaceSFX!*), add the SurfaceSFX assets to the **Player Controller's surface list**.

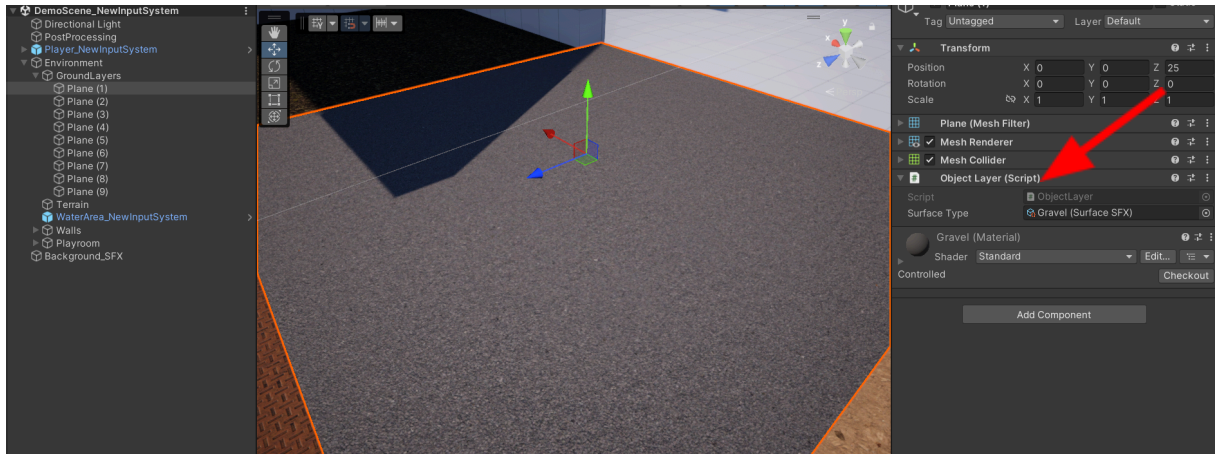


MFPC will automatically detect the active terrain layer under the player and play the correct sounds.

Using surface sounds on objects

To assign surface sounds to individual objects:

1. Add the **ObjectLayer** component to any GameObject
2. Assign a **SurfaceSFX** asset to the **surfaceType** field



When the player walks or lands on that object, MFPC will use the assigned surface sounds instead of terrain sounds.

This makes it easy to handle:

- Props
- Platforms
- Modular level pieces
- Dynamic or moving objects

Fallback behavior

If no matching surface is found:

- MFPC automatically falls back to a **generic surface preset**
- Water audio is handled separately when the player is marked as being in water

This ensures audio always plays correctly, even in mixed or incomplete setups.

Design philosophy

The audio system is designed to be:

- Modular
- Data-driven

- Easy to extend
- Free of hardcoded surface checks
- Compatible with both terrain-based and object-based levels

No layers, tags, or naming conventions are required.

New in v1.3

MFPC now includes a **snow** surface layer/preset as part of the surface-based audio workflow.

Like the other surface presets, snow audio is handled through **SurfaceSFX** and can be assigned through:

- terrain layers
- object-based surface setup
- fallback/general sound workflows if needed

No controller changes are required to use it.

8. Extending MFPC

MFPC is designed to be extended without modifying the core controller. The controller exposes key runtime states and supports optional external modules for animation, events, and camera behavior.

Public state access

The controller exposes key runtime states such as:

- `MoveInput`
- `LookInput`
- `Velocity`
- `IsGrounded`
- `VerticalVelocity` (“falling” velocity on y axis)
- `IsMoving`
- `IsSprinting`
- `IsCrouching`
- `IsAiming`
- `IsLeftLeaning`
- `IsRightLeaning`
- `AirborneTime`
- `InWater`

These can be used by:

- Animators
- UI systems
- Gameplay logic
- Audio or VFX controllers
- Custom scripts

Animator support

MFPC does not require an Animator, but it can drive one through an optional helper component.

Component: `AnimatorBridge`

`AnimatorBridge` reads controller state and `CharacterController` velocity, then maps them to Animator parameters.

Supported parameter types:

- Horizontal movement speed
- Grounded state

- Crouch state
- Sprint state
- Aim state
- Falling state
- Jumping state
- Vertical velocity

Features:

- Supports both `PlayerController_NewInputSystem` and `PlayerController_Legacy`
- Optional automatic controller detection
- Optional automatic Animator lookup in children
- Can update in `Update` or `LateUpdate`
- Uses actual `CharacterController.velocity` for speed and vertical motion

Typical use cases:

- Driving idle / walk / run blend trees
- Triggering jump and fall transitions
- Syncing crouch and aim animation layers
- Keeping animation logic external to the controller

This approach preserves MFPC's system-agnostic design while making it easy to integrate character animation.

Event hooks

MFPC also includes an optional event-driven extension:

Component: `EventObserver`

`EventObserver` monitors runtime state transitions (such as *leave ground*, *land*, *sprint start / end*, *crouch start / end*, *aim start / end*) and it exposes UnityEvents for common gameplay hooks.

Available events:

- `OnAirborneStart`
- `OnLand`
- `OnSprintStart`
- `OnSprintEnd`
- `OnCrouchStart`
- `OnCrouchEnd`
- `OnAimStart`
- `OnAimEnd`
- `OnMoveStart`

- `OnMoveEnd`
- `OnLeanLeftStart`
- `OnLeanLeftEnd`
- `OnLeanRightStart`
- `OnLeanRightEnd`

Both controllers now expose: `OnJumpStart`, which is invoked when the player actively performs a jump.

Features:

- Supports both input system controller variants
- Optional automatic controller detection
- Uses transition-based state checks rather than continuous polling logic in external scripts
- Optional debug logging for transition tracing

This is useful for:

- Audio triggers
- VFX playback
- UI feedback
- Gameplay reactions
- Animator or timeline integration
- Designer-friendly event hookup directly from the Inspector

By exposing these transitions as UnityEvents, MFPC stays modular while making common integrations much easier.

Modular by design

MFPC extensions are implemented as optional modules, not baked-in systems.

Examples include:

- `AnimatorBridge` for Animator parameter syncing
- `EventObserver` for UnityEvent-based state transitions
- `NoiseHeadbobController` for optional camera noise disabling

This keeps the controller focused while allowing developers to add only the systems they need.

9. What MFPC does *not* include (by design)

MFPC focuses on delivering a **clean and extensible controller core**.

Some systems are intentionally **not included** to avoid forcing architectural decisions.

MFPC does **not** include built-in implementations for:

- Multiplayer or networking logic
- Animation-driven movement systems
- Advanced traversal mechanics (wall-running, vaulting, sliding, etc.)s

These systems vary significantly between projects.

MFPC is designed to **integrate with them**, not restrict or replace them.

Optional helpers such as **AnimatorBridge** and **event hooks** are provided to support these integrations.

10. Optional Modules

MFPC includes all the following optional additional modules:

- **SlopeHandler**
Corrects movement on slopes.
- **SteepSlopeSlideModule**
Forces downhill sliding on slopes above a configurable walkable angle, with optional camera tilt feedback.
- **StaminaModule**
Adds sprint drain, jump cost, regeneration, and exhaustion behavior. The demo stamina UI is included only as an example and can be removed entirely.
- **AnimatorBridge**
Pushes controller state into an Animator.
- **EventObserver**
Exposes state transitions through UnityEvents.
- **NoiseHeadbobController**
Disables Cinemachine noise/headbob globally at startup when requested by settings.
- **LeanDutchModule**
Applies Cinemachine Dutch roll based on the controller's current lean angle.

This modular structure lets developers add only the systems they actually need.

11. Roadmap

MFPC follows a clear, incremental roadmap focused on **stability first, features second**.

v1.1 — “Integration Update” (Completed and soon available)

Focus: smoother integration with common project setups.

- Optional Animator Bridge (separate component)
- Public event hooks (e.g. `OnJump`, `OnLand`, `OnSprintStart`)
- Minor slope handling improvements
- API cleanup
- Separation of Toggle/Hold mode for crouching and sprinting
- Headbob disable option
- URP, HDRP import fixes
- Unity 6 compatibility fix

v1.2 - Control & Feedback Update

Included:

- optional `StaminaModule`
- sprint drain and recovery
- jump stamina cost
- gamepad look multiplier
- global lean enable/disable option
- new and improved public event hooks
- public extension properties for movement state access
- custom Cinemachine camera blends for smoother state transitions
- Youtube tutorials (importing and sound layers customization)

v1.3 — “Movement Expansion”

Included:

- Camera leaning improvements
- Optional `SteepSlopeSlideModule`
- Camera tilt feedback on steep slopes
- Optional `LeanDutchModule` for Cinemachine Dutch roll while leaning

- Advanced headbob blend/profile presentation
- New snow sound layer / preset
- Additional documentation updates
- Additional Youtube tutorials (controller customization)

v1.3.1 — Cinemachine 3+ fix

Included:

- Cinemachine 3+ missing script fix

v1.4+ — Community-driven

Focus: features guided by real usage.

- Community-voted features
- Presets and quality-of-life improvements
- Accessibility options

The roadmap is public and may evolve based on feedback and real-world use cases.

12. Support & community

MFPC is actively maintained and supported.

Community & support channels

A dedicated [Discord server](#) is available for:

- Bug reports
- Feature requests and polls
- Usage questions
- Roadmap discussions

Community feedback directly influences future updates and priorities.

You can find useful youtube tutorials on my channel here: [Youtube tutorials](#).

MFPC is built as a long-term asset, not a one-off release.